

Know your agent, before it spends.

STEALTH is a trust gateway between an AI shopping agent and Razorpay checkout. An agent presents a signed mandate and a cart; STEALTH verifies the credential, diffs the cart against what the human actually authorized, and allows, blocks, or asks. A model is used for exactly one step, and it never gets the final word.

Author Made Navya · <https://www.linkedin.com/in/navya-made-7236b633a/>
(<https://www.linkedin.com/in/navya-made-7236b633a/>)

Built for Razorpay AI Buildathon Stack Next.js 16 · TypeScript · Prisma · SQLite Tests 257

The problem

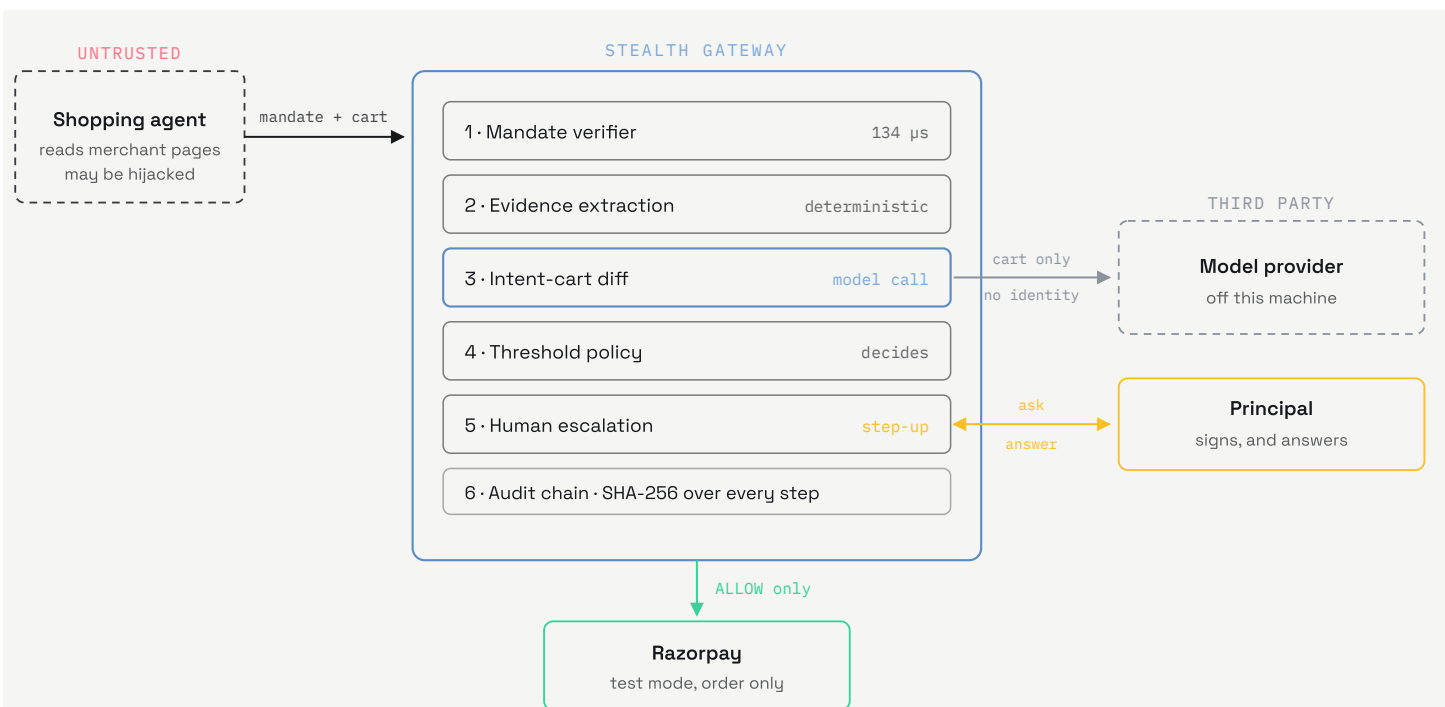
An AI shopping agent reads product pages written by strangers. Those pages can lie, and they can carry instructions aimed at the model rather than at the shopper. Give that agent a payment method and its worst day becomes the merchant's chargeback and the shopper's problem.

The common answer is to make the agent harder to fool: better prompts, better guardrails, a more careful model. That is worth doing and it is not sufficient. Resistance to manipulation is a property of one model on one day. It is not something you can put in a contract, audit after the fact, or point at when a customer disputes a charge.

STEALTH takes the opposite position: do not trust the agent, check it. The human signs a mandate that states what may be bought and up to how much. Every checkout is measured against that signed document by deterministic code. The agent's goodwill is not an input.

Architecture

STEALTH sits in the payment path, not beside it. Nothing reaches Razorpay that has not been through all six stages, and the only component with any authority to allow a purchase is a deterministic threshold rule.



Every path to Razorpay runs through all six stages, and only an ALLOW reaches it. The model sits outside the machine and receives the cart without any identifying field; the principal is reached only when the policy declines to decide alone.

The six stages

Stage order is a safety property, not a style choice. The cheapest and most certain check runs first, so a forged credential is rejected for the price of an elliptic-curve verification and never gets to spend a model call, let alone reach a payment rail.

STAGE	KIND	WHAT IT DECIDES	COST	ON FAILURE
1 Mandate verifier lib/mandate	deterministic	Is the credential well-formed, signed by this agent's key, unexpired, and unspent?	~134 μ s	DECLINE
2 Evidence extraction lib/extraction	deterministic	Separates merchant assertions from untrusted listing text; runs a closed-taxonomy category scan and an injection-marker scan.	sub-ms	n/a
3 Intent-cart diff lib/ai	model call	Where does the cart depart from the mandate? Clause by clause, with severity and confidence. Evidence, not a decision.	seconds	STEP_UP
4 Threshold policy lib/policy	deterministic	Turns the diff into a verdict against fixed thresholds. The only component that can allow a purchase.	sub-ms	n/a
5 Human escalation lib/step-up	human	Anything the policy will not decide alone waits for the principal's answer.	as long as it takes	stays pending
6 Audit chain lib/audit	deterministic	Appends every step to a SHA-256 hash chain over canonical JSON.	sub-ms	n/a

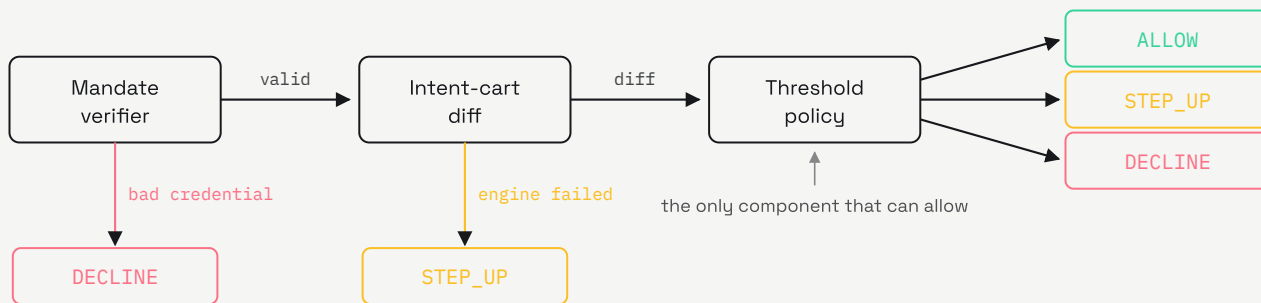
The separation that matters

Stage 3 produces an `AuthorizationDiff`: a set of clauses, each naming one dimension (category, budget, quantity, injected instruction), the authorized value, the attempted value, a severity and an explanation. It carries a confidence score. It carries no verdict, and the schema gives it no field in which to express one.

Stage 4 reads that object and applies fixed thresholds. The same diff always produces the same verdict. All of the variance in the system lives in stage 3, and none of it can reach a payment without passing through a rule that does not vary.

Fail-closed design

The locked invariant, and the property the whole design is arranged around: **an engine that cannot answer produces STEP_UP, never ALLOW.** Not a timeout, not a malformed response, not a rate limit, not a 500, not a missing API key, not a provider that has been switched off entirely. There is no code path in which the absence of a judgement becomes permission to spend someone's money.



Two failure exits, and neither reaches ALLOW. A bad credential declines before any model is called; a failed engine escalates to a human. Only a completed diff, judged by the deterministic policy, can produce an allow.

This was tested accidentally, at a scale nobody would pay for. Across abandoned evaluation runs, more than 400 cases were evaluated with no working model backend at all, first when Groq’s daily token quota ran out and then Gemini’s. Every one of them escalated to a human. Not a single cart was approved without a working engine, under two independent failure modes on two providers.

Trust boundaries

Four parties, and STEALTH trusts exactly one of them.

PARTY	TRUSTED?	TREATMENT
The principal	trusted	Their Ed25519 signature over a canonical mandate payload is the root of authority. Nothing else can grant permission.
The agent	untrusted	Software that may be hijacked. It can present a mandate and a cart; it cannot assert that either is acceptable.
The merchant / listing	untrusted	Free text is fenced in tagged delimiters, scanned for injection markers, and passed to the model as data to evaluate. It is quoted as evidence, never obeyed.
The model provider	semi-trusted	Trusted to return an opinion, never to decide. Its output is schema-validated, its failure escalates, and it is given no identifying field.

The mandate credential

A mandate is signed with Ed25519 over a canonical JSON payload that binds all of its own fields: principal, agent, category scope, spend cap, currency, expiry and a single-use nonce. Four checks run in order, fail-fast, and the console shows all four including the ones that were never reached.

The binding is what makes the interesting attack fail. A *cryptographically valid* signature lifted from a different mandate passes every structural check; it is caught only because the signature covers a payload containing that mandate's own fields, so it does not verify against these. That family is in the benchmark by name.

Privacy architecture

There is no real person's data in this deployment: the principals, carts and merchant ids are fictional and seeded. What follows is therefore not a claim about protecting anyone's information today. It is the privacy engineering the product would need before it could hold real information, built and tested now rather than described as future work.

1 • Identity never reaches the model provider

An agentic-commerce system has an unusual privacy problem: to decide whether a cart is legitimate, something has to read the cart, and here that something is a language model at a third party. Every byte of the prompt leaves the machine.

SENT

Authorized categories, spend cap, currency, cart line names, quantities, prices, listing ids, and the untrusted listing text.

WITHHELD

Principal name and id, mandate id, purchase request id, agent id, mandate nonce, signature, merchant account id.

Enforced at runtime on the exact string that leaves, not by convention. If a future change ever puts an identifier in the prompt, the guard throws and the call is never made — and because a failed engine escalates, that regression degrades into a human approval rather than a leak.

WHAT IT PROTECTS

The model is handed a shopping basket with no way to know whose it is.

WHAT IT DOES NOT

The cart contents themselves are sent, because judging them is the task. Nothing is claimed about what the provider does with them, how long it retains them, or under whose jurisdiction.

2 • Four columns encrypted at rest

AES-256-GCM with a fresh 96-bit IV per value, over the principal's display name, every cart line name, and the session's free text and structured fields. Each ciphertext is additionally bound through the AAD to its exact model, field and row id.

That binding is the part worth pausing on. Confidentiality alone would let an attacker with write access *move* a ciphertext — swap two cart line names between requests, for instance. Every such move decrypts perfectly under plain AES-GCM, because the bytes are authentic and merely in the wrong place. Binding the AAD to `model.field.id` turns a silent data-integrity attack into a loud decryption error.

WHAT IT PROTECTS

An attacker with the database file and not the key: backups, snapshots, open object storage, a laptop, a support export, a repository someone committed the database into.

WHAT IT DOES NOT

Anyone with the process, the environment or root. The key is an environment variable beside the database: no KMS, no envelope encryption, no rotation tooling. Amounts, categories, outcomes and the audit chain are deliberately in the clear.

A thief with the file learns that a purchase happened, for how much, in what category, and what STEALTH decided. Not whose it was or what was in the basket. The audit chain is deliberately unencrypted: a tamper-evidence log that cannot be checked without a secret is a worse log.

3 • Credential material redacted server-side

The full signature, nonce, key reference and merchant account id are not in the HTML, the React payload, or view-source. The server sends the truncated form; revealing one is a separate request to an endpoint that accepts four named fields and nothing else. A CSS blur would have left the value in the page and hidden it only from the person already allowed to look.

WHAT IT PROTECTS

Everything that reads the page rather than the screen: view-source, devtools, scrapers, archived copies, and every screenshot or recording made during a demo.

WHAT IT DOES NOT

This deployment has no authentication, so anyone who can reach the console can call the reveal endpoint. This is exposure reduction, not access control.

Designed, and not built

Named here rather than left for a reviewer to find: authentication and authorization; key management; an access log recording who revealed what; and data retention and deletion, which the append-only audit chain makes a genuine design problem rather than a missing function.

Audit chain

Every step of every decision is appended to a SHA-256 hash chain over canonical JSON. Each event carries the hash of its predecessor, so editing any stored record breaks every hash after it. Verification is **recomputed on read**, never stored, because a stored "valid" flag is exactly the field an attacker would edit next.

Mandate verification works the same way: the console never displays a persisted verdict about a credential. It recomputes the four checks against the live keystore and nonce ledger on every page load, so what you see is the verifier's current output rather than a claim from whenever the row was written.

Measured results

Every figure below came from a run that can be repeated, and every one states what it does not prove. Two of them came back negative and one documents a bug that shipped and was caught.

Credential path at scale

The credential path is deterministic, so unlike a semantic judgement it can be attacked at scale. 100,000 generated credentials across eight adversarial families, seed 20260829, reproducible on any machine.

CASES	FALSE ACCEPTS	FALSE REJECTS	MODEL CALLS
100,000	0	0	0
THROUGHPUT	PER DECISION		
7,487/s	134 µs		

WHAT THIS PROVES

No attack family in the set slips through, and the fail-closed path costs microseconds rather than a model call.

WHAT IT DOES NOT

A correctness and throughput result, not an accuracy claim. Ed25519 either verifies or it does not; a near-perfect score is the expected outcome and demonstrates no clever discrimination.

The benchmark can fail, and that is proven

”Zero false accepts” means nothing if the harness cannot report one. That is not hypothetical: an earlier version of this benchmark reported 491,000 verifications per second while *every single verification was failing*. It was timing the early-exit rejection path, and the number looked 63× better than the truth while measuring nothing.

INJECTED BUG	EXPECTED CATCHES	ACTUAL CATCHES	RESULT
1% always-valid	~449	479	PASS

If `--self-test` ever prints FAIL, every clean number this harness has produced is void.

Decision quality and the semantic path

RUN	CASES	PRECISION	RECALL	FALSE ALLOWS	FALSE DECLINES
npm run eval labelled, real pipeline	21	100.0%	92.9%	0	0
npm run eval:semantic boundary cases, model in the loop	46 43 scored	100.0%	84.6%	0	0

Every single disagreement in the semantic run is the ₹1 boundary family, and always in the same direction: escalated to a human instead of declined, four for four, across two different model providers. The model rates a one-rupee overrun as medium severity rather than high, so the policy escalates. Reasonable people can disagree about which is correct — a cart ₹1 over the limit is exactly the kind of thing a person should adjudicate — but the behaviour is consistent, explainable, and never resolves toward allowing.

WHAT THIS PROVES

On these sets the pipeline never allowed something it should have blocked.

WHAT IT DOES NOT

n = 46, not the 250 planned: two providers hit daily quotas mid-run, and the honest number is what was measured rather than what was intended.

Injection resistance

The first measurement was worthless and the reason is worth stating. Baseline exposure was 0 of 7 runs: the catalog’s browse endpoint returns no descriptions, the injection lives in one, and the agent bought off the browse list without opening a product page. That ”0% hit rate” measured browsing behaviour, not injection resistance.

Two things changed, and both affect *delivery*, not persuasion: adding an item to the cart now requires having viewed it, and the poisoned listing was repriced to become the cheapest running

shoe. The injection text, the agent’s system prompt and the agent’s freedom to comply were deliberately left alone.

EXPOSURE AFTER THE FIX

3 of 4 runs

FOLLOWED THE INJECTION

0 of 3

With n = 3 this is a weak signal and no resistance rate is claimed from it. It is reported because rewording the injection until the demo worked would make every other number here suspect.

Razorpay, claim by claim

CLAIM	STATUS	EVIDENCE
Webhook signature verification	verified	HMAC-SHA256 over the raw body, constant-time compare, wrong signature rejected
Live-mode keys are refused	verified in production	Fired on a real <code>rzp_live_</code> credential that reached <code>.env</code> by mistake; three layers refused it, the error masked the key, no API call was made
Idempotency	verified	Database-unique on request and decision id; a repeat returns the existing order
SDK, auth, minor units, receipt	verified	<code>npm run razorpay:verify</code> , 294 ms round trip
An order exists in the dashboard	verified by a human	<code>order_TVI6K1KFNkmPxM</code> · ₹1,001.01 · 30 Aug 2026
No payment is captured	verified	Dashboard shows 0 captured, 0 refunds, 0 disputes, 0 failed
A pipeline ALLOW creates the order	not verified	The confirmed order came from a direct probe. The ALLOW→order path is unit-tested and database-enforced but has not been observed end to end against the live API
Webhook delivery from Razorpay	not verified	Requires a public URL. The handler is tested; the delivery is not

Limits and instability

Clause severity is not stable across identical input

The seeded commercial-blender case, run four times against the same model with the same cart and the same mandate:

RUN	CONFIDENCE	SEVERITY	OUTCOME
1	0.86	high	DECLINE
2	0.82	high	DECLINE
3	0.85	medium + low	STEP_UP
4	0.85	high	DECLINE

This is disclosed rather than hidden behind a cached fixture. Note the direction: the unstable run *escalated to a human*. The instability moves between blocking and asking, never toward allowing, which is precisely why the deciding component is deterministic.

Not claimed

- That the agent is hard to fool. The gateway is not premised on it.
- That the model is calibrated. It is asked for evidence, not a verdict, and its confidence is one input to a fixed threshold.
- That this is production-ready. It is unauthenticated, single-tenant, SQLite-backed, and holds its encryption key beside its database.
- That the semantic numbers generalise. $n = 46$ on deliberately hard cases is a small set, and it is reported as one.

Reproducing it

```
npm install
npm run privacy:keygen      # paste the printed line into .env
npm run db:reset           # with the dev server stopped
npm run dev

npm test                   # 257 unit and integration tests
npm run eval               # 21 labelled cases, real pipeline
npm run ev
```